

Reviewer Pack — Minimal Rank of Etale Cohomology over Boolean Tensor Product...

Entry 0974b2abdfec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 11:39:30 UTC

Minimal Rank of Etale Cohomology over Boolean Tensor Product Valuations

Entry ID: 0974b2abdfec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 11:39:30 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Etale Cohomology) **Field B** (complexity object): Communication Complexity (Boolean Tensor Product)

Statement:

{‘sentence_1’: ‘For any given boolean function f with n variables, the minimal rank of its associated etale cohomology group $E(f)$ is upper bounded by the maximum number of independent clauses in the boolean tensor product valuation of f .’, ‘sentence_2’: ‘This upper bound holds for all instances where the function is represented as a disjunctive normal form (DNF) and the boolean tensor product valuation is computed over $\text{GF}(2)$.’, ‘sentence_3’: ‘In particular, $E[f]$ has rank at most $\max(0, |\text{CNF}(f)| - 1)$, where $\text{CNF}(f)$ denotes the number of clauses in the DNF representation of f .’}

Rationale (proposer’s reasoning):

{‘sentence_1’: “The etale cohomology group captures non-trivial algebraic properties of the function’s underlying structure, which might be related to its communication complexity.”, ‘sentence_2’: “Boolean tensor product valuations are a well-studied tool in communication complexity, offering a way to measure the information needed to communicate a function’s value.”, ‘sentence_3’: ‘By exploring the interplay between these two areas, we may uncover new insights into the structure of boolean functions and their computational properties.’}

Taxonomy category: ETALE_COHOMOLOGY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ebd627f8dc3c67a7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of etale cohomology group $E(f)$ for a boolean function f is supported if, across 30 random seeds, the Spearman rank correlation coefficient between the etale cohomology ranks and the maximum clause counts is ≥ 0.7 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "etale cohomology" AND "Boolean tensor product valuation" AND minimal rank" - "disjunctive normal form" AND etale cohomology AND boolean function" - "communication complexity" AND "maximum independent clauses" AND etale cohomology group

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def generate_boolean_function(n: int) -> str:
    clauses = []
    for _ in range(random.randint(1, n)):
        clause = [random.choice(['x' + str(i) if i < 26 else 'x' + chr(97 + i - 26) for i in range(n)]]
        clauses.append(' | '.join(clause))
    return ' & '.join(clauses)

def compute_etale_cohomology_rank(f: str) -> int:
    # Placeholder function to simulate etale cohomology computation
    # This is a dummy implementation for the purpose of this example
    return len(f.split(' & '))

def max_clause_count(f: str) -> int:
    return max(len(clause.split(' | ')) for clause in f.split(' & '))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    etale_cohomology_rank = compute_etale_cohomology_rank(f)
    max_clause_count_val = max_clause_count(f)

    return {
        "metric_name": "etale_cohomology_rank",
        "metric_value": etale_cohomology_rank,
        "instances_tested": 1,
        "conjecture_holds": etale_cohomology_rank <= max_clause_count_val,
```

```

        "counterexample": "" if conjecture_holds else f"etale_cohomology_rank={etale_cohomology_rank}"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.7:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c46c6ecb.py", line 53, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c46c6ecb.py", line 45, in run_trial
    "counterexample": "" if conjecture_holds else f"etale_cohomology_rank={etale_cohomology_rank}",
                        ^^^^^^^^^^^^^^^^^^^^^
NameError: name 'conjecture_holds' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13238
2	preregistration	ollama_remote	glm4:latest	0	0	6325
3	novelty	ollama_remote	glm4:latest	0	0	4772
4	novelty	ollama_remote	glm4:latest	0	0	6766
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40862
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7039
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8528
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7795
9	judge	ollama_remote	glm4:latest	0	0	29733

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125056 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0974b2abdfec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0974b2abdfec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0974b2abdfec.tar.gz` (if generated)